Add catalog item reviews

eShopOnContainers (открытый проект) · тестовый MR · Фича «отзывы о товарах»: модель, EF-конфигурация, сервис, контроллер и тесты
автор: разработчик · ветка: feature/catalog-reviews · коммитов: 2

98%

Вероятность, что код написан **Claude Code**

Почти наверняка написан с помощью Claude Code
Надёжность: средняя–высокая — 420 строк кода, 15 независимых групп сигналов, есть сравнение с кодом проекта

Код похож на работу Claude Code: за ИИ говорят: async и токены отмены, коммиты и процесс, именование; за человека: форматирование; устойчивые к маскировке стиля группы за ИИ: async и токены отмены, коммиты и процесс, форма MR, тесты.

Главные признаки ИИ

+1.0 CancellationToken принимается почти всеми async-методами Async и токены отмены
9 из 9 async-методов принимают CancellationToken (в коде проекта: 0%)

+0.7 Тесты размечены // Arrange / // Act / // Assert Тесты
5 из 6 тестов

```
src/Services/Catalog/Catalog.UnitTests/Application/CatalogReviewServiceTest.cs:33
public async Task AddReviewAsync_ValidRequest_PersistsReview()
```

```
src/Services/Catalog/Catalog.UnitTests/Application/CatalogReviewServiceTest.cs:48
public async Task AddReviewAsync_SameBuyerTwice_ThrowsCatalogDomainException()
```

+0.6 Частое использование nameof(...) Современный синтаксис и API
5 раз на 319 строк

+0.6 Токен отмены передаётся дальше по всей цепочке вызовов Async и токены отмены
9 из 9 await-вызовов получают токен

+0.5 Токен отмены всегда назван полностью: cancellationToken Именование
9 из 9 параметров CancellationToken

+0.4 Код в основном в новых файлах — написан целиком Форма MR
8 новых из 10 C#-файлов

+0.4 Новые файлы написаны «с первого раза»: каждый создан одним коммитом и больше не правился Коммиты и процесс
6 новых файлов, в среднем 1.0 коммита на файл, 100% созданы одним коммитом

+0.4 Сообщения коммитов «как по учебнику»: императив/Conventional Commits, тело со списком изменений Коммиты и процесс
2 коммитов, средняя оценка 0.70

```
коммит 6ef43e40
Add catalog item reviews to Catalog.API
```

```
коммит 9697534d
Add unit tests for CatalogReviewService
```

+0.4 Нет «ручных» следов: TODO, закомментированного кода, #region, Console.WriteLine, string.Format Ручные привычки
0 на 319 строк кода

+0.4 **Очень высокая скорость: сотни строк кода за минуты между коммитами** Коммиты и процесс
коммит 9697534d: +111 строк через 7 мин после предыдущего (~444 строк/час)

Признаки человека

-0.4 **Небрежности форматирования (пробелы в конце строк, if(...), x=y, двойные пустые строки)** Форматирование
6 небрежностей на 319 строк (в проекте 2.7 на 100 строк)

```
src/Services/Catalog/Catalog.API/Controllers/CatalogReviewsController.cs:31
public async Task<IActionResult> GetReviewsAsync(int catalogItemId, [FromQuery] int pageSize = 10, [FromQuery] int pa...
```

-0.3 **Новые возможности C# почти не используются** Современный синтаксис и API
0 конструкций на 319 строк (0.0 на 100 строк), в проекте 0.0

-0.2 **Классы не запечатаны (нет sealed)** Структура классов
0 из 6 новых конкретных классов (в коде проекта: 0%)

-0.2 **XML-doc неполный (нет <param>/<returns>)** XML-документация
0 из 3 документированных методов с параметрами

-0.2 **Строковые свойства без инициализации** Современный синтаксис и API
0 из 4 строковых автосвойств (в коде проекта: 0%)

-0.1 **«Магические числа» прямо в коде** Структура классов
10 литералов в выражениях, 1 констант

-0.1 **Новые файлы с блочным namespace { }** Современный синтаксис и API
0 из 8 новых файлов (в коде проекта: 0%)

-0.1 **Стиль сильно различается между файлами MR (неровно, как у человека)** Форма MR
4 файлов, разброс оценок 0.45

Файлы (оценка только по коду)

84% `src/Services/Catalog/Catalog.API/Services/CatalogReviewService.cs` +107
CancellationToken принимается почти всеми async-методами; Токен отмены передаётся дальше по всей цепочке вызовов

67% `src/Services/Catalog/Catalog.API/Services/ICatalogReviewService.cs` +28
CancellationToken принимается почти всеми async-методами; XML-doc почти на каждом публичном члене

63% `src/Services/Catalog/Catalog.API/Controllers/CatalogReviewsController.cs` +72
CancellationToken принимается почти всеми async-методами; Токен отмены передаётся дальше по всей цепочке вызовов

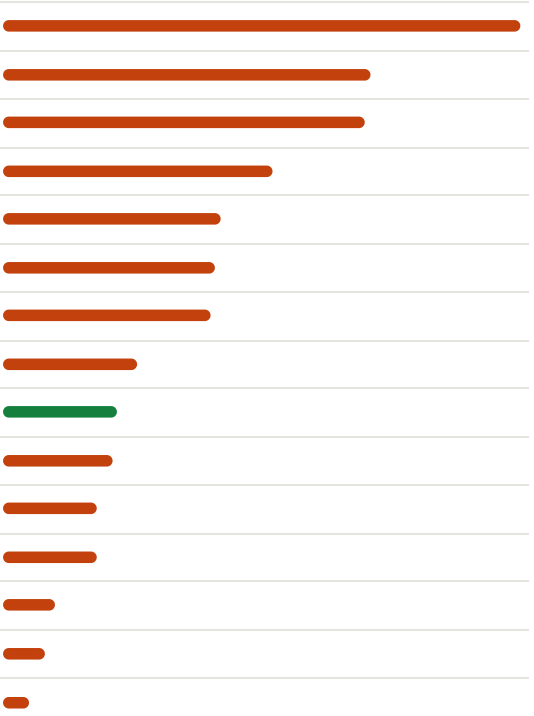
58% `src/Services/Catalog/Catalog.UnitTests/Application/CatalogReviewServiceTest.cs` +111
Тесты размечены // Arrange / // Act / // Assert; Нет «ручных» следов: TODO, закомментированного кода, #region, Console.WriteLine, string.Format

46% `src/Services/Catalog/Catalog.API/Model/CatalogItemReview.cs` +29
Код MR заметно «аккуратнее» и современнее существующего кода проекта; Шаблонные формулировки XML-doc («Gets or sets...», «A token to cancel...»)

43% `src/Services/Catalog/Catalog.API/Infrastructure/EntityConfigurations/CatalogItemReviewEntityTypeConfiguration.cs` +38
Параметры лямбд осмысленные (order ⇒ order.Id / o ⇒ o.Id)

Вклад групп сигналов

Async и токены отмены устойчив к маскировке	+2.0
Коммиты и процесс устойчив к маскировке	+1.4
Именование	+1.4
Форма MR устойчив к маскировке	+1.0
Современный синтаксис и API	+0.8
Тесты устойчив к маскировке	+0.8
Исключения	+0.8
Структура классов	+0.5
Форматирование	-0.4
Ручные привычки	+0.4
Отличие от стиля проекта	+0.4
Комментарии	+0.4
Защитный код	+0.2
Логирование устойчив к маскировке	+0.2
XML-документация	+0.1



Проанализировано: 10 C#-файлов из 10 изменённых; +309 строк кода, +111 строк тестов; база проекта: 16 файлов.
Оценка вероятностная и не является доказательством авторства.

Folder cleanup

bitwarden/server (открытый проект) · тестовый MR · Очистка пустых и дублирующихся папок: сервис, эндпоинты, модель ответа и тесты
автор: разработчик · ветка: feature/folder-cleanup · коммитов: 2

93%

Вероятность, что код написан **Claude Code**

Почти наверняка написан с помощью Claude Code
Надёжность: средняя–высокая — 259 строк кода, 14 независимых групп сигналов, есть сравнение с кодом проекта

Код похож на работу Claude Code: за ИИ говорят: коммиты и процесс, async и токены отмены, форма MR; устойчивые к маскировке стиля группы за ИИ: коммиты и процесс, async и токены отмены, форма MR, логирование, тесты.

Главные признаки ИИ

+0.9 CancellationToken принимается почти всеми async-методами Async и токены отмены
6 из 6 async-методов принимают CancellationToken (в коде проекта: 0%)

+0.5 Частое использование nameof(...) Современный синтаксис и API
5 раз на 205 строк

+0.4 Токен отмены всегда назван полностью: cancellationToken Именование
6 из 6 параметров CancellationToken

+0.4 Код в основном в новых файлах — написан целиком Форма MR
5 новых из 6 C#-файлов

+0.4 Сообщения коммитов «как по учебнику»: императив/Conventional Commits, тело со списком изменений Коммиты и процесс
2 коммитов, средняя оценка 0.70

```
коммит 8c7ba029
Add folder cleanup analysis and empty folder removal
```

```
коммит 932ae45f
Add FolderCleanupService tests
```

+0.4 Нет «ручных» следов: TODO, закомментированного кода, #region, Console.WriteLine, string.Format Ручные привычки
0 на 205 строк кода

+0.4 Новые файлы написаны «с первого раза»: каждый создан одним коммитом и больше не правился Коммиты и процесс
4 новых файлов, в среднем 1.0 коммита на файл, 100% созданы одним коммитом

+0.4 Тесты написаны в том же MR, что и код Форма MR
+69 строк тестов к +190 строкам кода

+0.4 Структурное логирование: шаблоны {OrderId} вместо интерполяции Логирование
2 из 2 вызовов логгера с параметрами — через шаблон

```
src/Core/Services/Implementations/FolderCleanupService.cs:55
_logger.LogInformation(
```

```
src/Core/Services/Implementations/FolderCleanupService.cs:89
_logger.LogInformation("Deleted {DeletedCount} empty folders for user {UserId}", deleted, userId);
```

+0.3 **Тесты размечены // Arrange / // Act / // Assert** Тесты

2 из 3 тестов

```
test/Core.Test/Services/FolderCleanupServiceTests.cs:31
public async Task AnalyzeAsync_FolderWithoutCiphers_IsReportedAsEmpty()
```

```
test/Core.Test/Services/FolderCleanupServiceTests.cs:48
public async Task AnalyzeAsync_FoldersWithSameName_AreGroupedAsDuplicates()
```

Признаки человека

-0.4 **Токен отмены принимается, но не передаётся дальше** Async и токены отмены

3 из 7 await-вызовов получают токен

-0.3 **Мало типовых имён (result, existing..., normalized...)** Именование

0 из 20 локальных переменных

-0.2 **Нет параметризованных тестов** Тесты

0 из 3 тестов, 0 наборов данных

-0.1 **Новые возможности C# почти не используются** Современный синтаксис и API

0 конструкций на 205 строк (0.0 на 100 строк), в проекте 0.0

-0.1 **Классы не запечатаны (нет sealed)** Структура классов

0 из 3 новых конкретных классов (в коде проекта: 0%)

-0.1 **Тестов немного относительно кода** Форма MR

соотношение тесты/код = 0.36

-0.1 **Новые файлы с блочным namespace { }** Современный синтаксис и API

0 из 5 новых файлов (в коде проекта: 0%)

Файлы (оценка только по коду)

69% `src/Core/Services/Implementations/FolderCleanupService.cs` +102

CancellationToken принимается почти всеми async-методами; Частое использование nameof(...)

65% `src/Core/Services/IFolderCleanupService.cs` +29

CancellationToken принимается почти всеми async-методами; XML-doc почти на каждом публичном члене

50% `test/Core.Test/Services/FolderCleanupServiceTests.cs` +69

Тесты размечены // Arrange / // Act / // Assert; Нет «ручных» следов: TODO, закомментированного кода, #region, Console.WriteLine, string.Format

44% `src/Core/Models/Api/Response/Folders/FolderCleanupResponseModel.cs` +26

Guard-проверки аргументов в начале публичных методов

Вклад групп сигналов

Коммиты и процесс устойчив к маскировке	**+1.0**	
Async и токены отмены устойчив к маскировке	**+1.0**	
Форма MR устойчив к маскировке	**+1.0**	
Исключения	**+0.5**	
Именование	**+0.5**	
Современный синтаксис и API	**+0.5**	

Ручные привычки	+0.4	
XML-документация	+0.4	
Логирование устойчив к маскировке	+0.4	
Структура классов	+0.3	
Тесты устойчив к маскировке	+0.3	
Защитный код	+0.3	
Форматирование	+0.2	
Отличие от стиля проекта	+0.1	

Проанализировано: 6 C#-файлов из 6 изменённых; +190 строк кода, +69 строк тестов; база проекта: 15 файлов.
Оценка вероятностная и не является доказательством авторства.

⚠ Учтены конвенции проекта (соответствующие признаки ослаблены): .editorconfig со стилевыми правилами.

Wishlist

eShopOnWeb (открытый проект) · тестовый MR · Список желаний, написанный «в стиле проекта»: без XML-doc, без разметки тестов, короткие коммиты
автор: разработчик · ветка: feature/wishlist · коммитов: 2

72%

Вероятность, что код написан **Claude Code**

Скорее написан с помощью Claude Code
Надёжность: средняя–высокая — 270 строк кода, 12 независимых групп сигналов, есть сравнение с кодом проекта

Код похож на работу Claude Code: за ИИ говорят: async и токены отмены, именование, структура классов; за человека: тесты, форматирование, XML-документация; устойчивые к маскировке стиля группы за ИИ: async и токены отмены, форма MR.

Главные признаки ИИ

+0.8 CancellationToken принимается почти всеми async-методами Async и токены отмены
6 из 6 async-методов принимают CancellationToken (в коде проекта: 56%)

+0.5 Токен отмены передаётся дальше по всей цепочке вызовов Async и токены отмены
7 из 7 await-вызовов получают токен

+0.5 Код в основном в новых файлах — написан целиком Форма MR
7 новых из 7 C#-файлов

+0.4 Частое использование nameof(...) Современный синтаксис и API
3 раза на 229 строк

+0.4 Нет «ручных» следов: TODO, закомментированного кода, #region, Console.WriteLine, string.Format Ручные привычки
0 на 229 строк кода

+0.4 Все приватные поля readonly Структура классов
9 из 9 приватных полей readonly

+0.4 Новые файлы написаны «с первого раза»: каждый создан одним коммитом и больше не правился Коммиты и процесс
4 новых файлов, в среднем 1.0 коммита на файл, 100% созданы одним коммитом

+0.4 Тесты написаны в том же MR, что и код Форма MR
+61 строк тестов к +209 строкам кода

+0.3 Суффикс Async у всех асинхронных методов Async и токены отмены
6 из 6 async-методов (кроме действий контроллеров) (в коде проекта: 83%)

+0.3 Токен отмены всегда назван полностью: cancellationToken Именование
7 из 7 параметров CancellationToken (в коде проекта: 100%)

Признаки человека

-0.6 Тесты без разметки Arrange/Act/Assert Тесты
0 из 3 тестов

-0.5 **Небрежности форматирования (пробелы в конце строк, if(...), x=y, двойные пустые строки)** Форматирование

3 небрежностей на 229 строк (в проекте 0.6 на 100 строк)

```
tests/UnitTests/ApplicationCore/Services/WishlistServiceTests/AddItem.cs:24
_mockWishlistRepo.Setup(x => x.FirstOrDefaultAsync(It.IsAny<WishlistWithItemsSpecification>(),
It.IsAny<CancellationT...
```

-0.4 **Проверки на null через == null / != null** Современный синтаксис и API

0 из 2 проверок на null — через is

```
src/ApplicationCore/Entities/WishlistAggregate/Wishlist.cs:39
if (item != null)
```

```
src/ApplicationCore/Services/WishlistService.cs:32
if (wishlist != null)
```

-0.3 **Публичные члены почти без XML-doc** XML-документация

0 из 31 публичных классов/методов/свойств с XML-doc (в коде проекта: 3%)

-0.3 **Лямбды с безликими параметрами x ⇒ x...** Именование

71% лямбд с параметрами x/y/a/b...

-0.3 **Новые возможности C# почти не используются** Современный синтаксис и API

0 конструкций на 229 строк (0.0 на 100 строк), в проекте 0.0

-0.2 **Сообщения коммитов короткие и неформальные** Коммиты и процесс

2 коммитов, средняя оценка -0.40

```
коммит cbc7e19c
wishlist
```

```
коммит f230f97b
tests for wishlist
```

-0.2 **Классы не запечатаны (нет sealed)** Структура классов

1 из 7 новых конкретных классов (в коде проекта: 20%)

Файлы (оценка только по коду)

69% `src/ApplicationCore/Services/WishlistService.cs` +60

CancellationToken принимается почти всеми async-методами; Токен отмены передаётся дальше по всей цепочке вызовов

43% `src/ApplicationCore/Entities/WishlistAggregate/Wishlist.cs` +45

Guard-проверки аргументов в начале публичных методов; IReadOnlyList/IReadOnlyCollection в сигнатурах

39% `src/PublicApi/WishlistEndpoints/AddToWishlist.cs` +59

Нет «ручных» следов: TODO, закомментированного кода, #region, Console.WriteLine, string.Format; Идеальное форматирование: ни одного пробела в конце строки, if(...) без пробела, двойной пустой строки

35% `tests/UnitTests/ApplicationCore/Services/WishlistServiceTests/AddItem.cs` +61

Все приватные поля readonly; Нет «ручных» следов: TODO, закомментированного кода, #region, Console.WriteLine, string.Format

Вклад групп сигналов

Async и токены отмены устойчив к маскировке	**+1.8**	
Тесты устойчив к маскировке	**-0.9**	
Именование	**+0.7**	
Структура классов	**+0.6**	
Форма MR устойчив к маскировке	**+0.6**	

Форматирование	-0.5	
Ручные привычки	+0.4	
XML-документация	-0.3	
Коммиты и процесс устойчив к маскировке	+0.3	
Логирование устойчив к маскировке	+0.2	
Современный синтаксис и API	-0.1	
Отличие от стиля проекта	+0.1	
Защитный код	+0.1	

Проанализировано: 7 C#-файлов из 7 изменённых; +209 строк кода, +61 строк тестов; база проекта: 14 файлов.
Оценка вероятностная и не является доказательством авторства.